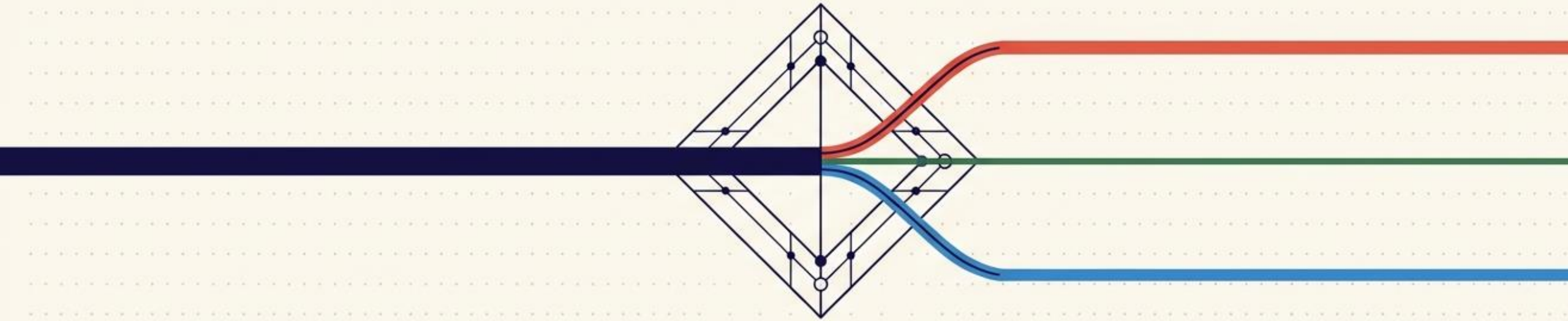


Concurrent Execution with Java Threads



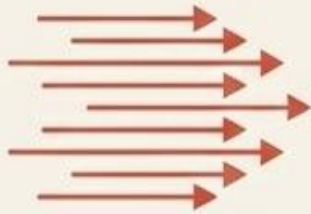
A blueprint for building, running, and
managing lightweight subprocesses.

Thread

Multithreading in Java is a feature that enables a program to run multiple threads simultaneously, allowing tasks to execute in parallel and utilize the CPU more efficiently.

A thread is a lightweight, independent unit of execution inside a program (process).

Why Implement Multithreading?



Improved Performance

Multiple threads execute tasks concurrently, drastically reducing overall processing time.



Better Resource Utilization

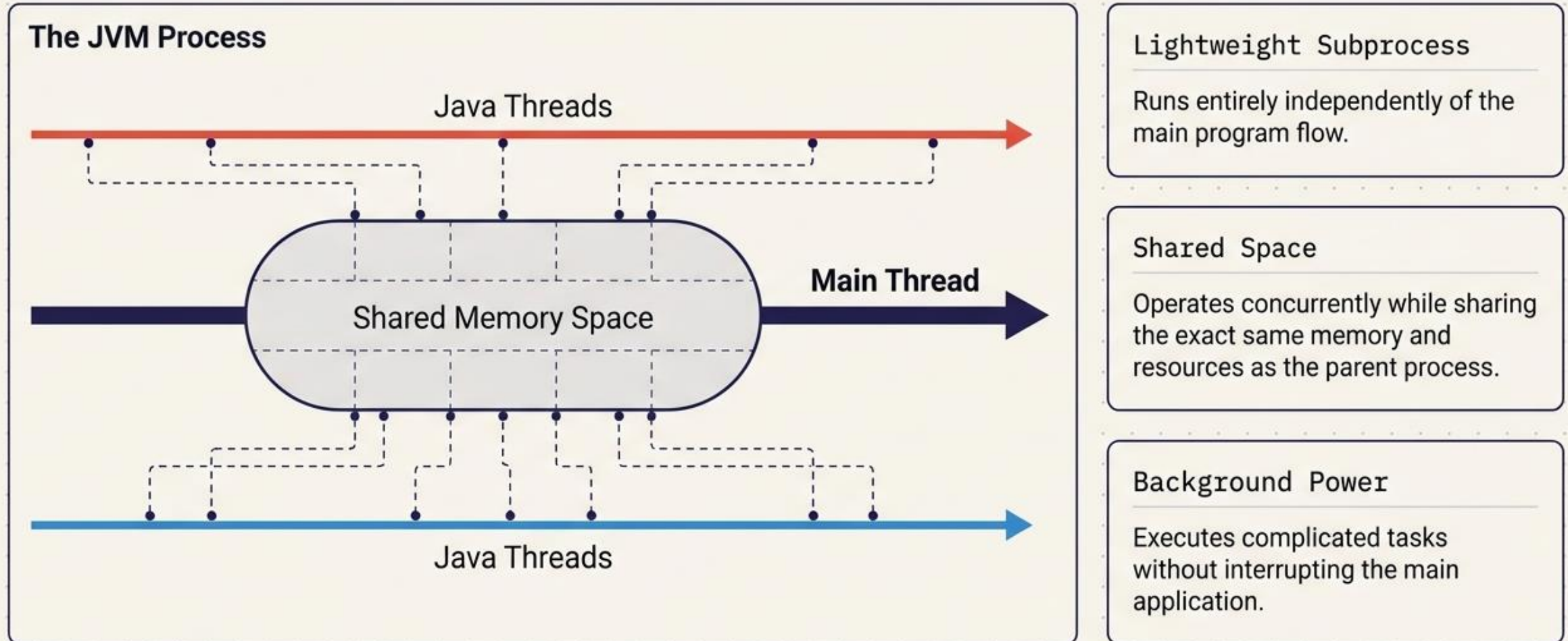
Because threads share the same memory space, they require less overhead than spinning up entirely new processes.



Responsive Applications

UI applications remain completely responsive to users while threads perform heavy computing in the background.

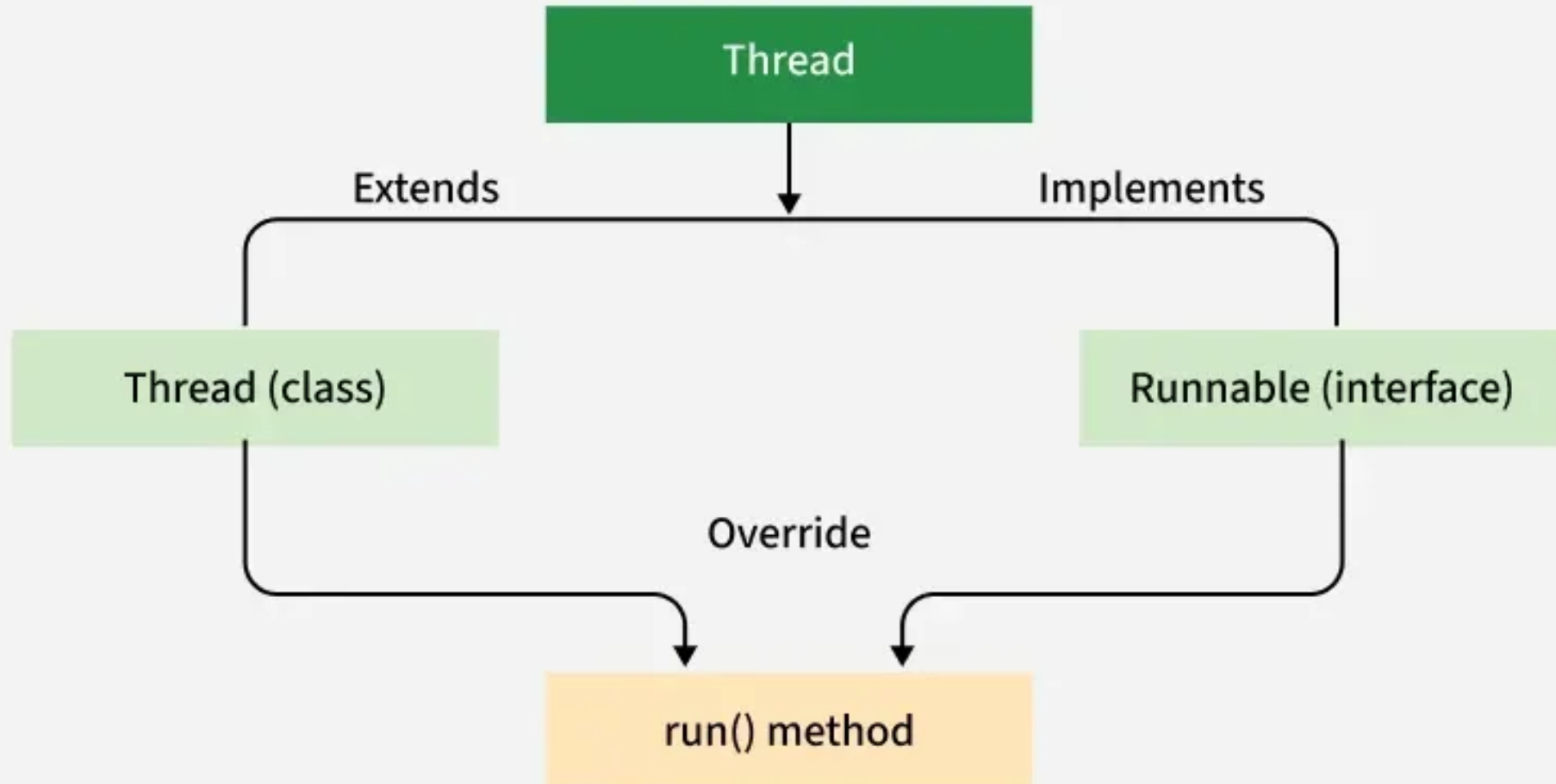
The Smallest Unit of Execution



Two Paths of Thread Creation

extends Thread	implements Runnable
Create a class that extends the Thread class.	Create a class that implements the Runnable interface.
Override the run() method.	Pass the Runnable object to a Thread constructor.
Limitation: You cannot extend any other class due to Java's single inheritance rule.	Advantage: Extremely flexible. Allows extending another class simultaneously. Preferred in most cases.

Thread-Creation



Thread-Creation

```
public class Main extends Thread {  
    public void run()  
    {  
        System.out.println("This code is running in a thread");  
    }  
}
```

```
public class Main implements Runnable {  
    public void run()  
    {  
        System.out.println("This code is running in a thread");  
    }  
}
```

If the class extends the `Thread` class, the thread can be run by creating an instance of the class and call its `start()` method:

Extend Example

```
public class Main extends Thread {  
    public static void main(String[] args) {  
        Main thread = new Main();  
        thread.start();  
        System.out.println("This code is outside of the thread");  
    }  
    public void run() {  
        System.out.println("This code is running in a thread");  
    }  
}
```


If the class implements the `Runnable` interface, the thread can be run by passing an instance of the class to a `Thread` object's constructor and then calling the thread's `start()` method:

Implement Example

```
public class Main implements Runnable {  
    public static void main(String[] args) {  
        Main obj = new Main();  
        Thread thread = new Thread(obj);  
        thread.start();  
        System.out.println("This code is outside of the thread");  
    }  
    public void run() {  
        System.out.println("This code is running in a thread");  
    }  
}
```

Extending the Thread class example

We create a class that extends Thread and override its [run\(\)](#) [method](#) to define the task. Then, we make an object of this class and call [start\(\)](#), which automatically calls run() and begins the thread's execution.

- We created multiple threads (t1–t4) using the CookingTask class.
- Each thread represents a dish being prepared.
- Calling start() creates a new thread with its own call stack and internally invokes the run() method. This allows threads to run concurrently with the [main thread](#) and each other.

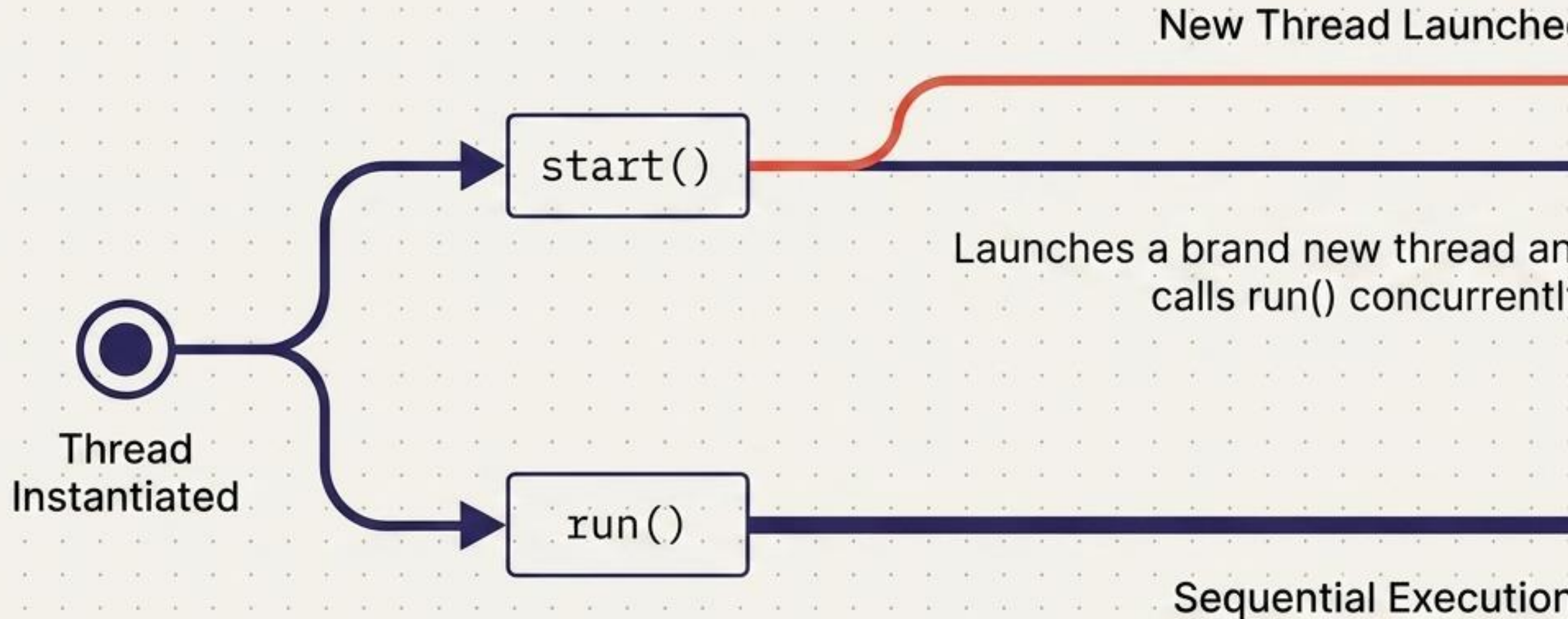
```
class CookingTask extends Thread {  
    private String task;  
  
    CookingTask(String task) {  
        this.task = task;  
    }  
  
    public void run() {  
        System.out.println(task + " is being  
prepared by " +  
        Thread.currentThread().getName());  
    }  
}  
  
public class Restaurant {  
    public static void main(String[] args) {  
        Thread t1 = new CookingTask("Pasta");  
        Thread t2 = new CookingTask("Salad");  
        Thread t3 = new CookingTask("Dessert");  
        Thread t4 = new CookingTask("Rice");  
  
        t1.start();  
        t2.start();  
        t3.start();  
        t4.start();  
    }  
}
```

Note: *The order of thread execution may vary on each run because thread scheduling is non-deterministic.*

Output

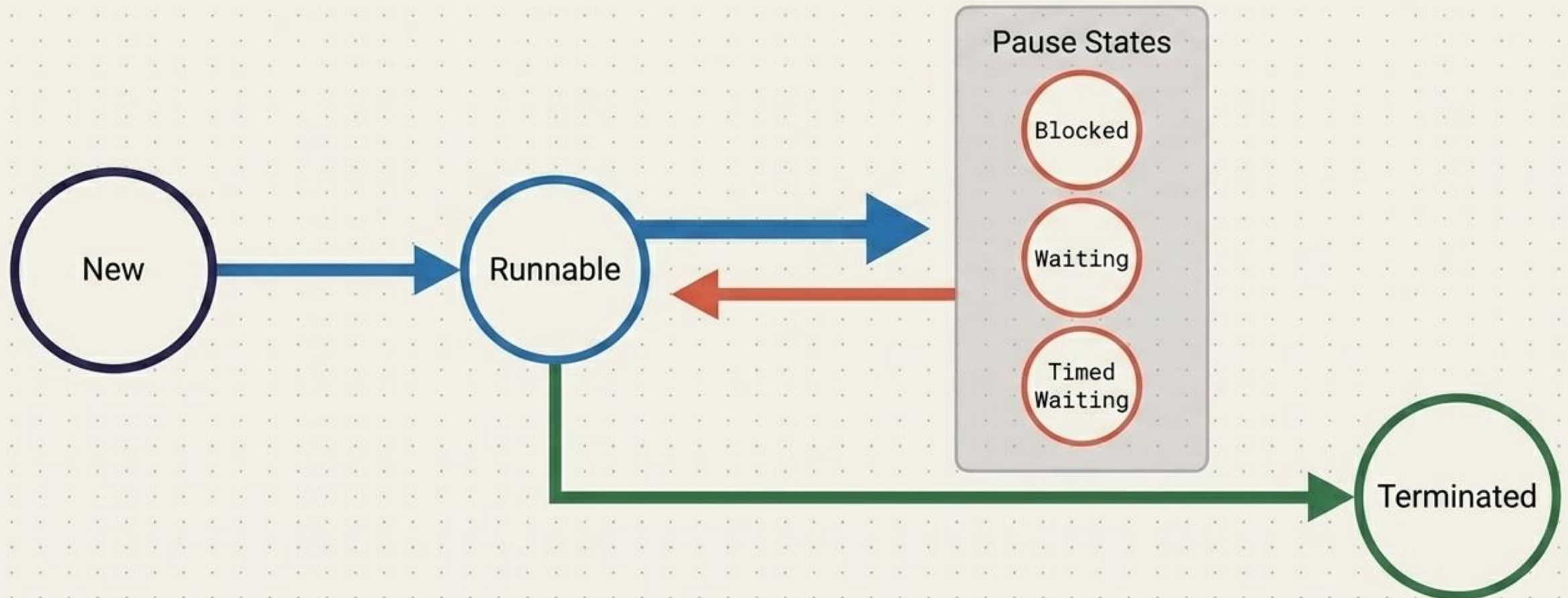
```
Rice is being prepared by Thread-3  
Pasta is being prepared by Thread-0  
Dessert is being prepared by Thread-2  
Salad is being prepared by Thread-1
```


Execution Logic: start() vs. run()



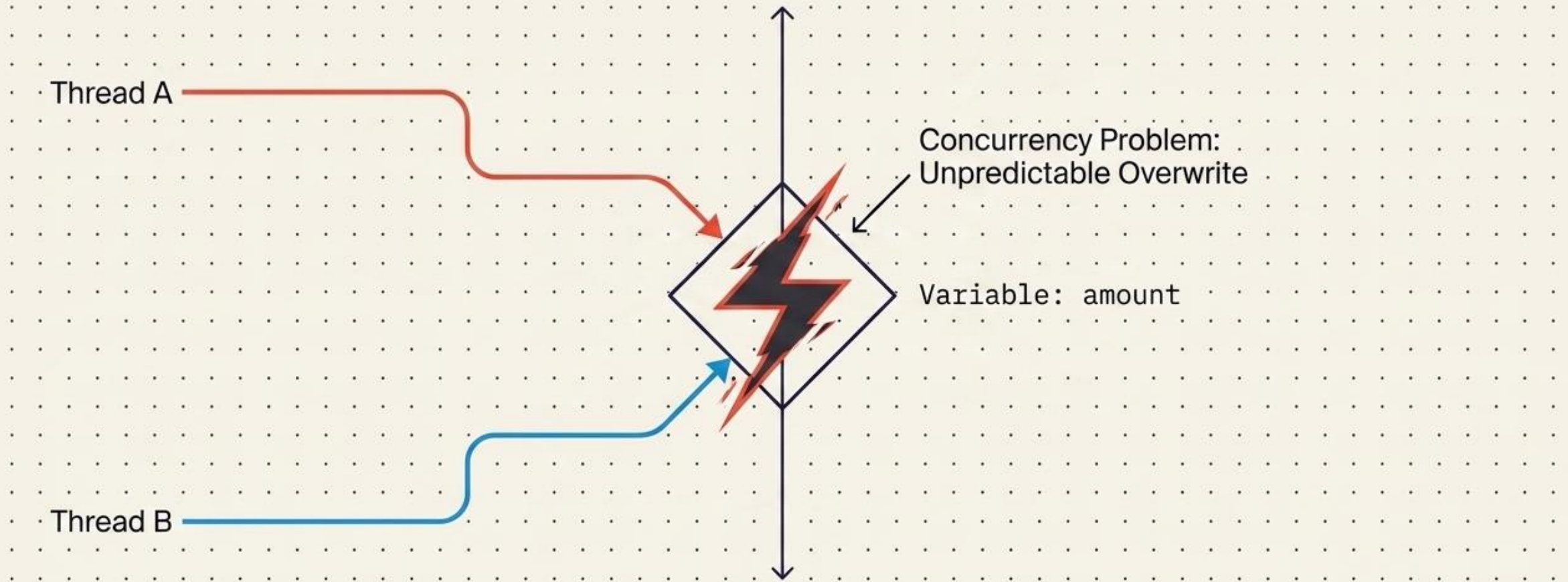
The Thread Lifecycle

A Java thread transitions through these precise states from creation to termination. Execution happens in the Runnable state, while memory locks or deliberate pauses push the thread into Blocked or Waiting states until resolved.



The Danger of Shared Memory

Because threads run simultaneously, execution order is inherently unknown. When parallel threads attempt to read and write to the exact same variables at the exact same time, the resulting values become unpredictable and corrupted.



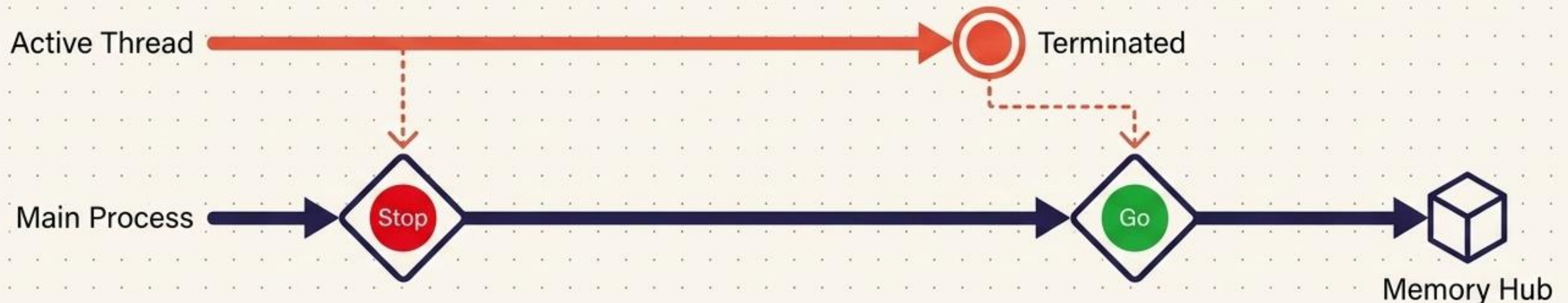
Preventing Concurrency Glitches

Rule 1: Minimize Sharing.

The best defense is to share as few attributes between threads as possible.

Rule 2: The `isAlive()` Check.

If attributes must be shared, use the thread's `isAlive()` method as a check-valve. The main program must wait to ensure the thread has completely finished running before touching any attributes the thread can change.



Rule 1: Minimize Sharing Between Threads

Bad Example (Shared Variable → Race Condition)

```
class BankAccount {  
  
    int balance = 100;  
  
    public void withdraw(int amount) {  
        if (balance >= amount) {  
            System.out.println(Thread.currentThread().getName() + " is withdrawing...");  
  
            balance = balance - amount;  
  
            System.out.println(Thread.currentThread().getName() + " new balance: " + balance);  
        } else {  
            System.out.println("Not enough balance");  
        }  
    }  
}
```

Rule 1: Minimize Sharing Between Threads

Bad Example (Shared Variable → Race Condition)

```
public class BadExampleForSharedResInThreads {  
    public static void main(String[] args) {  
  
        BankAccount account = new BankAccount();  
  
        Thread t1 = new Thread() -> {  
            account.withdraw(80);  
        }, "Thread-1");  
  
        Thread t2 = new Thread() -> {  
            account.withdraw(80);  
        }, "Thread-2");  
  
        t1.start();  
        t2.start();  
    }  
}
```

Possible Output

Thread-2 is withdrawing...
Thread-1 is withdrawing...
Thread-1 new balance: -60
Thread-2 new balance: 20

Rule 1: Minimize Sharing Between Threads

Solve the problem

```
class WithdrawalTask extends Thread {  
    private int amount;  
    private int result;  
  
    public WithdrawalTask(int amount) {  
        this.amount = amount;  
    }  
    public void run() {  
        // Thread only stores its withdrawal request  
        result = amount;  
        System.out.println(Thread.currentThread().getName() +  
            " wants to withdraw: " + result);  
    }  
  
    public int getResult() {  
        return result;  
    }  
}
```

Rule 1: Minimize Sharing Between Threads

Solve the problem

```
public class BankExample {  
  
    public static void main(String[] args) throws InterruptedException {  
  
        int balance = 100;  
  
        WithdrawalTask t1 = new WithdrawalTask(80);  
        WithdrawalTask t2 = new WithdrawalTask(80);  
  
        t1.start();  
        t2.start();  
  
        // Wait for threads to finish  
        t1.join();  
        t2.join();  
    }  
}
```

Rule 1: Minimize Sharing Between Threads

Solve the problem

```
// Main thread processes withdrawals safely
int totalWithdraw = t1.getResult() + t2.getResult();

if (balance >= totalWithdraw) {
    balance -= totalWithdraw;
} else {
    System.out.println("Not enough balance for both withdrawals");
}

System.out.println("Final balance: " + balance);
}
}
```

Possible Output

Thread-1 wants to withdraw: 80

Thread-0 wants to withdraw: 80

Not enough balance for both withdrawals

Final balance: 100

Rule 2: Solving the bank race condition using `isAlive()`

```
class BankAccount {  
    int balance = 100;  
  
    public void withdraw(int amount) {  
        if (balance >= amount) {  
            System.out.println(Thread.currentThread().getName() + " withdrawing " + amount);  
            balance = balance - amount;  
            System.out.println("Remaining balance: " + balance);  
        } else {  
            System.out.println("Not enough balance");  
        }  
    }  
}
```

Rule 2: Solving the bank race condition using `isAlive()`

```
class WithdrawThread extends Thread {  
  
    BankAccount account;  
  
    WithdrawThread(BankAccount account) {  
        this.account = account;  
    }  
  
    public void run() {  
        account.withdraw(80);  
    }  
}
```

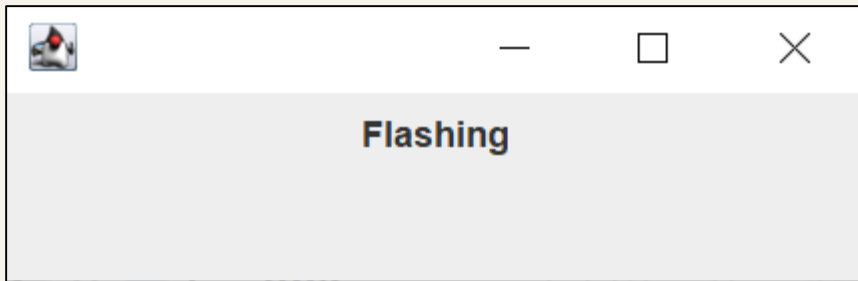
Possible Output

Thread-0 withdrawing 80
Remaining balance: 20
Not enough balance

```
public class IsAliveWithdrawThreadBankExample {  
  
    public static void main(String[] args) {  
  
        BankAccount account = new BankAccount();  
  
        WithdrawThread t1 = new WithdrawThread(account);  
        WithdrawThread t2 = new WithdrawThread(account);  
  
        t1.start();  
  
        // Wait until thread 1 finishes  
        while (t1.isAlive()) {  
            // do nothing, just wait  
        }  
  
        // Start second thread only after first finishes  
        t2.start();  
    }  
}
```


Example: Flashing Text

```
import javax.swing.*;
class CurrentThreadDemo extends JFrame{
    static JLabel label=new JLabel("Flash");
    public CurrentThreadDemo(){
        setSize(300,100);
        JPanel p=new JPanel();
        p.add(label);
        add(p); setVisible(true);}
}
```



```
public static void main(String args[]) {
    new CurrentThreadDemo();
    try {
        for(int n = 100; n > 0; n--) {
            if (label.getText()==null)
                label.setText("Flashing");
            else label.setText(null);
            Thread.sleep(500);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Interrupts

- An *interrupt* is an indication to a thread that it should stop what it is doing and do something else.
- It's up to the programmer to decide exactly how a thread responds to an interrupt, but it is very common for the thread to terminate.
- Many methods that throw **InterruptedException**, such as `sleep`, are designed to cancel their current operation and return immediately when an interrupt is received.
- If the thread is frequently invoking methods that throw **InterruptedException**, it simply returns from the `run` method after it catches that exception

Interrupts

```
class WorkerThread extends Thread {  
  
    public void run() {  
        try {  
            while (true) {  
                System.out.println("Thread working...");  
  
                // Simulate work  
                Thread.sleep(2000);  
            }  
        } catch (InterruptedException e) {  
            System.out.println("Thread was interrupted!");  
        }  
  
        // Thread exits after interruption  
        System.out.println("Thread terminating...");  
    }  
}
```

Interrupts

```
public class InterruptExample {  
  
    public static void main(String[] args) throws InterruptedException {  
  
        WorkerThread worker = new WorkerThread();  
        worker.start();  
  
        // Let the thread run for 5 seconds  
        Thread.sleep(5000);  
  
        System.out.println("Main thread interrupts worker...");  
        worker.interrupt(); // Send interrupt signal  
    }  
}
```

Possible Output

Thread working...

Thread working...

Thread working...

Main thread interrupts worker...

Thread was interrupted!

Thread terminating...

Interthread Communication

- Java allows threads to **coordinate with each other** instead of constantly checking a condition (busy waiting). It is usually implemented using the methods:
 - **wait()** tells the calling thread **to give up the monitor** and go to sleep until some other thread enters **the same monitor** and calls **notify()** or **notifyAll**.
 - **notify()** wakes up a thread that called **wait()** on the same object.
 - **notifyAll()** wakes up all the threads that called **wait()** on the same object.
- These methods belong to the **Object class** and must be used inside **synchronized blocks**.
- A very common example is the **Producer–Consumer problem**.

Interthread Communication

```
class Q {  
    int n;  
    synchronized int get() {  
        System.out.println("Got: " + n);  
        return n;  
    }  
    synchronized void put(int n) {  
        this.n = n;  
        System.out.println("Put: " + n);  
    }  
}
```

```
Put: 1  
Got: 1  
Got: 1  
Got: 1  
Got: 1  
Got: 1  
Put: 2  
Put: 3  
Put: 4  
Put: 5  
Put: 6  
Put: 7  
Got: 7
```

```
class Producer implements  
Runnable {  
    Q q;  
    Producer(Q q) {  
        this.q = q;  
        new  
Thread(this, "Producer").start();  
    }  
    public void run() {  
        int i = 0;  
        while(true) {  
            q.put(i++);  
        }  
    }  
}
```

```
class Consumer implements  
Runnable {  
    Q q;  
    Consumer(Q q) {  
        this.q = q;  
        new Thread(this,  
"Consumer").start();  
    }  
    public void run() {  
        while(true) {  
            q.get();  
        }  
    }  
}
```

```
class PC {  
    public static void main(String args[]) {  
        Q q = new Q();  
        new Producer(q);  
        new Consumer(q);  
    }  
}
```


Interthread Communication

A correct implementation of a producer and consumer.

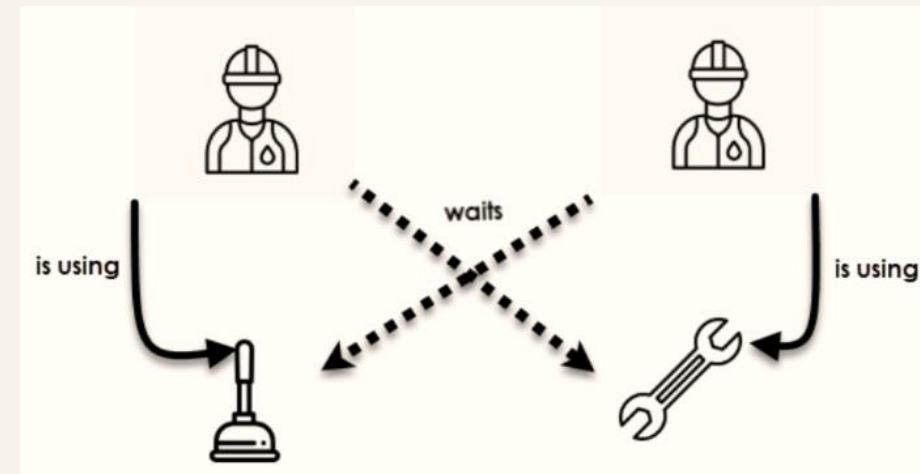
```
class Q {  
    int n;  
    boolean valueSet = false;  
    synchronized int get() {  
        if(!valueSet)  
            try {  
                wait();  
            } catch (InterruptedException e) {  
                System.out.println("InterruptedException caught");  
            }  
        System.out.println("Got: " + n);  
        valueSet = false;  
        notify();  
        return n;  
    }  
}
```

```
synchronized void put(int n) {  
    if(valueSet)  
        try {  
            wait();  
        } catch (InterruptedException e) {  
            System.out.println("InterruptedException caught");  
        }  
    this.n = n;  
    valueSet = true;  
    System.out.println("Put: " + n);  
    notify();  
}
```

Put: 1
Got: 1
Put: 2
Got: 2
Put: 3
Got: 3
Put: 4
Got: 4
Put: 5
Got: 5

Deadlock in Java Multithreading

- Deadlock is a situation in multithreading where two or more threads are permanently blocked because each one is waiting for the other to release a required lock. In simple terms, threads get stuck forever, and the program never continues.
- Each thread holds a lock and waits for another lock held by a different thread.
- This creates a circular wait, causing the application to freeze indefinitely.



Deadlock in Java Multithreading

```
class A {  
    synchronized void foo(B b) {  
        String name = Thread.currentThread().getName();  
        System.out.println(name + " entered A.foo");  
        try {  
            Thread.sleep(1000);  
        } catch (Exception e) {  
            System.out.println("A Interrupted");  
        }  
        System.out.println(name + " trying to call B.last()");  
        b.last();  
    }  
    synchronized void last() {  
        System.out.println("Inside A.last");  
    }  
}
```

```
class B {  
    synchronized void bar(A a) {  
        String name = Thread.currentThread().getName();  
        System.out.println(name + " entered B.bar");  
        System.out.println(name + " trying to call A.last()");  
        a.last();  
    }  
    synchronized void last() {  
        System.out.println("Inside A.last");  
    }  
}
```

Deadlock in Java Multithreading

```
class Deadlock implements Runnable {  
    A a = new A();  
    B b = new B();  
    Deadlock() {  
        Thread.currentThread().setName("MainThread");  
        Thread t = new Thread(this, "RacingThread");  
        t.start();  
        a.foo(b); // get lock on a in this thread.  
        System.out.println("Back in main thread");  
    }  
    public void run() {  
        b.bar(a); // get lock on b in other thread.  
        System.out.println("Back in other thread");  
    }  
    public static void main(String args[]) {  
        new Deadlock();  
    }  
}
```

Possible Output

RacingThread entered B.bar

MainThread entered A.foo

RacingThread trying to call A.last()

MainThread trying to call B.last()

Deadlock prevention

- Deadlock prevention is a strategy used in computer systems to ensure that different processes can run smoothly without getting stuck waiting for each other forever.
- As we have already discussed that deadlock can only happen if all four of the following conditions are met simultaneously:
 - Mutual Exclusion
 - Hold and Wait
 - No Preemption
 - Circular Wait

So we can prevent a Deadlock by eliminating any of the above four conditions

Deadlock prevention

- **Locks in Java**
- In Java, locks are mechanisms used to control access to shared resources in a multithreaded environment. Below is the diagrammatic representation of how Locks work and prevent Deadlock conditions.



Thanks

Which of the following describes a deadlock situation in Java multithreading?

- ☐ A When threads are not given enough CPU time
- ☐ B When two threads are blocked forever, each waiting for the other
- ☐ C When one thread keeps waiting for user input